# Camunda Compensate Event

In this session we will look at what is compensate event and how it functions ?.

- Compensation is undoing steps that were already successfully completed, because their results and possibly side effects are no longer desired and need to be reversed.
- If an activity is still active, it cannot be compensated, but rather needs to be canceled.
- Compensation is not an error handling
- A compensation handler performs the steps necessary to reverse the effects of an activity.
- When compensation is triggered, the compensation handler associated with the compensation boundary event is invoked the same number of times the activity it is attached to completed successfully.
- If a compensation boundary event is attached to an activity with multiple instance characteristics, a compensation event subscription is created for each instance.
- If a compensation boundary event is attached to an activity that is contained inside a loop, a compensation event subscription is created each time the activity is executed.
- If the process instance ends, the subscriptions to compensation events are canceled.

**Types of compensation events**

1. Compensation start event
2. Compensation end event
3. Compensation boundary event
4. Intermediate throw compensation event

**Compensation start event**

A compensation start event can only be used to trigger an Event Sub-Process - it cannot be used to start a process instance. This kind of event subprocess is called compensation event subprocess.

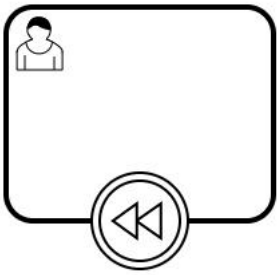
When deploying a process definition with a compensation event subprocess, the following considerations apply:

- The compensation event subprocess is only supported for embedded subprocess and not at process-level, caused by the current limitation that compensation is not propagated to sub process instances spawned by call activities.
- There can be only one compensation event subprocess at the same level of subprocess.
- A subprocess with a compensation event subprocess and an attached compensation boundary event is not supported. Note that the compensation event subprocess and the compensation boundary event have a similar intent, so only one of them should be chosen.

**Compensation end event**

A compensation end event triggers compensation and the current path of execution is ended. It has the same behaviour and limitations as a compensation intermediate throwing event.

**Compensation boundary event**



A compensation boundary event, can be used to attach a compensation handler to an activity or an embedded subprocess.

The compensation boundary event must reference a single compensation handler using a directed association.

A compensation boundary event has a different activation policy from other boundary events. Other boundary events, such as the signal boundary event, are activated when the activity they are attached to is started. When the activity is finished, they are deactivated and the corresponding event subscription is canceled. The compensation boundary event is different. The compensation boundary event is activated when the activity it is attached to completes successfully. At this point, the corresponding subscription to the compensation events is created. The subscription is removed either when a compensation event is triggered or when the corresponding process instance ends.

**Intermediate compensation throw event**

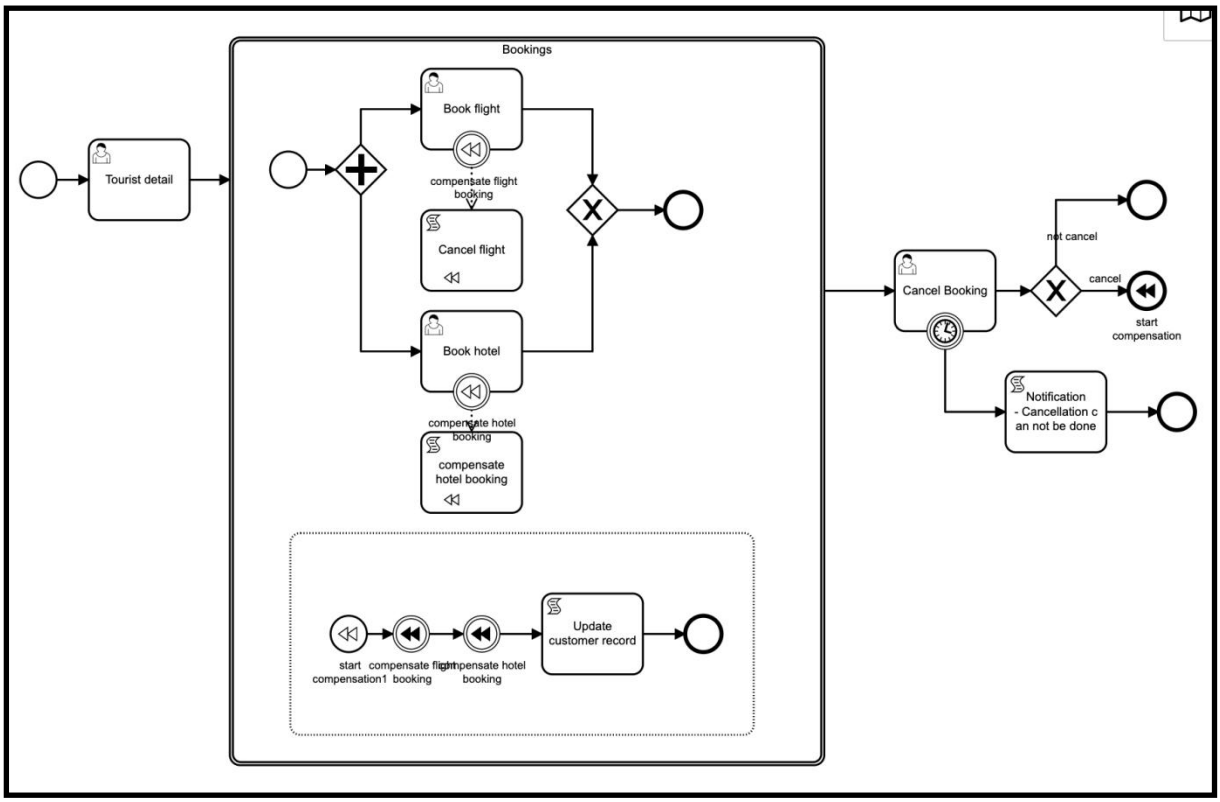
An intermediate throwing compensation event can be used to trigger compensation.

Triggering compensation: Compensation can either be triggered for a designated activity or for the scope that hosts the compensation event.

Compensation is performed through execution of the compensation handler associated with an activity.

- When compensation is thrown for an activity, the associated compensation handler is executed the same amount of times the activity completed successfully.
- If compensation is thrown for the current scope, all activities within the current scope are compensated, which includes activities on concurrent branches.
- Compensation is triggered hierarchically: if an activity to be compensated is a subprocess, compensation is triggered for all activities contained within the subprocess. If the subprocess has nested activities, compensation is thrown recursively. However, compensation is not propagated to the "upper levels" of the process: if compensation is triggered within a subprocess, it is not propagated to activities outside of the subprocess scope. The BPMN specification states that compensation is triggered for activities at "the same level of subprocess".
- Compensation is consumed by compensation event subprocess: if an activity to be compensated is a subprocess and the subprocess contains an event subprocess triggered by a compensation start event, compensation triggers the event subprocess instead of trigger the activities contained within the subprocess.
- Compensation is performed in reverse order of execution. This means that whichever activity completed last is compensated first, etc.
- The intermediate throwing compensation event can be used to compensate transaction subprocesses which completed successfully.

**To demonstrate the use of compensate events we will use following process diagram.**


Bookings
Tourist detail
Book flight
compensate flight booking
Cancel flight
Book hotel
compensate hotel booking
compensate hotel booking
start compensation1
compensate flight booking
compensate hotel booking
Update customer record
Cancel Booking
not cancel
cancel
start compensation
Notification - Cancellation c an not be done